Reproduction and Analysis of Low-Degree Polynomial Data Mappings via Linear SVM

Dally R
Roll Number: 230143

Advanced Machine Learning – Part B
NST, Rishihood University

March 2026

Abstract

This report reproduces the core method from Chang et al. (2010): explicit degree-2 polynomial feature mapping followed by linear SVM training. I implement the method on the 20 Newsgroups text classification dataset and find that the explicit mapped linear SVM achieves 87.37% accuracy, slightly outperforming the baseline polynomial kernel SVM (86.84%). Furthermore, I confirm the paper's claims regarding computational efficiency on sparse data, observing a roughly 18× training speedup (4.32s vs 77.31s). Through ablation studies, I show that cross-product interaction terms contribute significantly to the model's performance, while quadratic (squared) terms are essentially redundant. Finally, I demonstrate a failure mode on dense image data (MNIST) where the explicit mapping method becomes 6.4× slower than kernel SVM, violating the paper's sparsity assumption. These findings validate the paper's theoretical framework while highlighting its strict dependency on dataset scale and sparsity.

1 Paper Summary

Chang et al. (2010) tackle the computational bottleneck of training nonlinear SVMs by proposing an explicit computation of polynomial feature mappings instead of relying on traditional dual-optimization kernel methods. Standard kernel SVM requires evaluating $O(l \cdot \bar{n})$ operations per training iteration, where l is the number of training instances and $\bar{n}$ is the average number of nonzero features per instance. This scales poorly for large datasets.

By taking the explicit degree-2 polynomial mapping $\phi(\mathbf{x})$ and training a fast linear SVM (such as LIBLINEAR) on the transformed features, the per-iteration cost drops to $O(\hat{n})$, where $\hat{n}$ is the average number of nonzero features in the mapped space. Crucially, for highly sparse data, $\hat{n} \approx \bar{n}^2/2$. When this value is significantly smaller than $l \cdot \bar{n}$ (which occurs in large-scale, sparse domains like text classification), the explicit method delivers substantial speedups. The authors report 4× to over 4,600× faster training on various benchmark datasets (e.g., rcv1, news20, webspam) while maintaining classification accuracy comparable to Radial Basis Function (RBF) or polynomial kernel SVMs.

The objective is to train the explicit linear SVM by solving:

$$\min_{\mathbf{w}} \frac{1}{2}\mathbf{w}^T\mathbf{w} + C\sum_{i=1}^{l} \max(1 - y_i\mathbf{w}^T\phi(\mathbf{x}_i), 0)$$

For a degree-2 polynomial kernel $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^T\mathbf{z} + 1)^2$, the explicit mapping is formulated as:

$$\phi(\mathbf{x}) = [1, \sqrt{2}x_1, \ldots, \sqrt{2}x_n, x_1^2, \ldots, x_n^2, \sqrt{2}x_1x_2, \ldots, \sqrt{2}x_{n-1}x_n]^T$$

2 Reproduction Setup and Results

2.1 Dataset

For the reproduction, I selected the 20 Newsgroups dataset, re-cast as a binary classification problem (`comp`, `sci`, `talk` vs `alt`, `misc`, `rec`, `soc`). The dataset contains 18,846 total samples, split into 15,076 training samples and 3,770 test samples.

Using TF-IDF vectorization capped at 3,000 features with English stop words removed, the sparse text representation yielded an average of $\bar{n} = 37.8$ nonzero features per instance, corresponding to 98.74% sparsity. This firmly satisfies the paper's sparsity assumption ($\bar{n} \ll n$) and aligns closely with the paper's benchmark datasets (such as news20).

Limitations: The paper's fastest experiments utilized datasets with hundreds of thousands of samples (e.g., rcv1 with $l = 677,399$), while our dataset has $l = 15,076$. The speedups achieved here are more modest due to the lower scale but remain theoretically consistent.

2.2 Implementation

I implemented the mapping leveraging `scipy.sparse` and `scikit-learn`'s `PolynomialFeatures(degree=2, include_bias=False)`. The mapped data expanded from $n = 3,000$ to $4,504,500$ dimensions but remained highly sparse. I then trained a `LinearSVC` model with $C = 1.0$ and compared the results against the baseline polynomial `SVC(kernel='poly', degree=2)` and a plain `LinearSVC` without mapping.

2.3 Results

Table 1: Reproduction results on binary 20 Newsgroups (15,076 training samples)

Method	**Training Time (s)**	**Testing Time (s)**	**Accuracy (%)**
Explicit Poly + Linear SVM	4.32	0.3610	87.37
Polynomial Kernel SVM (Baseline)	77.31	6.4049	86.84
Plain Linear SVM	0.19	0.0016	86.31

2.4 Gap Analysis

Accuracy: The explicit mapping achieved 87.37% accuracy, which validates the method's ability to match or slightly exceed the baseline polynomial kernel SVM (86.84%) by reaching equivalent mathematical boundaries through primal coordinate descent versus sequential minimal optimization on the dual.

Training Time: The explicit mapping approach demonstrated an 18× speedup over the baseline kernel SVM (4.32s vs 77.31s). Testing time also saw an 18× speedup, as prediction shifted from evaluating kernel dot-products against thousands of support vectors to a single efficient vector dot-product $\mathbf{w}^T\phi(\mathbf{x})$. These results strongly support the central claims of the paper for sparse data regimes.

3 Ablation Study

To understand the internal dynamics of the mapped features, I performed component ablations on the $\phi(\mathbf{x})$ mapping.

3.1 Ablation 1: Removing Interaction Terms

I excluded all cross-product terms ($\sqrt{2}x_jx_k$ for $j \neq k$), retaining only the n linear and n quadratic terms.

Result: Accuracy dropped by 1.17 percentage points (from 87.37% to 86.21%). Training time decreased significantly to 0.17s.

Interpretation: Although individual word frequencies carry strong predictive signals, pairwise associations (e.g., "graphics" co-occurring with "monitor") provide essential nonlinear classification power. The absence of these interactions demonstrably weakens the model.

3.2 Ablation 2: Removing Quadratic Terms

I excluded all squared terms (x_j^2), retaining only the linear and interaction terms.

Result: Accuracy experienced a negligible 0.05 percentage-point drop (from 87.37% to 87.32%). Training time remained largely unchanged.

Interpretation: For normalized representations like TF-IDF, where values exist in $[0, 1]$, squared terms merely dampen existing values ($x_j^2 \leq x_j$) without capturing new relational information. For text classification, the performance of the degree-2 polynomial mapping is driven almost entirely by the linear and interaction features.

4 Failure Mode

4.1 Scenario

The paper explicitly states the necessity of data sparsity. To provoke a failure mode, I tested the implementation on a dense sub-sample of the MNIST image dataset (3 vs 8), where $n = 784$ and the average nonzero feature count is dense ($\bar{n} \approx 170$, $\sim 78\%$ dense).

4.2 Results

Table 2: Performance on dense MNIST data

Method	**Training Time (s)**	**Testing Time (s)**	**Accuracy (%)**
Explicit Poly + Linear SVM	1.7334	0.1472	98.00
Polynomial Kernel SVM (Baseline)	0.2654	0.1170	97.60
RBF Kernel SVM	0.3481	0.1982	97.20

4.3 Analysis

On dense data, the explicit mapping approach was 6.4× slower than the baseline polynomial kernel SVM (1.73s vs 0.27s). Due to the lack of sparsity, computing $\hat{n} \approx n^2/2$ causes the mapped dimensionality to explode to an intractable 308,505 sparse entries per instance. Computing, storing, and iterating over vectors of this size negates any advantage, violating Assumption 1 ($\bar{n} \ll n$).

Suggested fix: Applying PCA dimensionality reduction prior to explicit mapping. By reducing MNIST from 784 to 50 principal components, the mapped dimension shrinks to 1,325. Doing this yielded 98.20% accuracy while restoring training speeds (0.18s), proving that dimensionality compression circumvents the dense mapping explosion.

5 Reflection

5.1 What Surprised Me

The most surprising finding was the dramatic 18× training speedup achieved on the Newsgroups data relative to the baseline kernel SVM, strongly reflecting the hardware and algorithm efficiency when iterating over primal vs. dual forms when $\hat{n}$ is kept small via sparsity. I was also surprised by how redundant the quadratic terms proved to be during ablation; text books emphasize polynomial curvature, but text analytics overwhelmingly favor cross-feature associations.

5.2 What I Would Revisit

If time permitted, I would expand the analysis to include the multi-class dependency parsing application described in Section 5 of the paper, exploring how L1 regularization and sparse weight representations perform across a wider variety of algorithmic parameters. I would also experiment with a C++ implementation to bypass Python loop overheads when investigating higher-order degree-3 mappings.

5.3 Conclusion

I successfully reproduced the core methodology of Chang et al. (2010) and validated their theoretical claims. Explicit degree-2 polynomial mapping effectively matches polynomial kernel SVM accuracy (87.37% vs 86.84%) while providing massive training and testing speedups (18×) on large-scale sparse datasets. The ablation confirmed cross-product terms dominate performance gains, while the MNIST failure mode empirically proved the method's strict vulnerability to dense data representations. The paper's contribution remains fundamentally valuable for sparse classification tasks but is computationally inappropriate for dense feature spaces.

References

[1] Y.-W. Chang, C.-J. Hsieh, K.-W. Chang, M. Ringgaard, and C.-J. Lin, "Training and testing low-degree polynomial data mappings via linear SVM," *Journal of Machine Learning Research*, vol. 11, pp. 1471–1490, 2010.